Java Full Stack Development with React.js

Detailed Course Outline with Lab Manuals

120 Hours | 30 Sessions × 4 Hours | Instructor-Led | Freshers → Production-Ready Software Engineers

---

Phase Overview

Phase	Focus Area	Sessions	Hours	Technologies
Phase 1	**Foundations**	Day 1–5	20 hrs	Core Java, OOP, Collections, Git, JUnit 5
Phase 2	**Database**	Day 6–9	16 hrs	SQL, MySQL, DDL/DML, JOINs, PL/SQL
Phase 3	**Backend**	Day 10–17	32 hrs	Hibernate/JPA, Spring Boot, REST APIs, JWT Security
Phase 4	**Frontend**	Day 18–24	28 hrs	JavaScript ES6+, React.js, Redux Toolkit, Tailwind CSS
Phase 5	**Integration & Deploy**	Day 25–30	24 hrs	Docker, Jenkins CI/CD, Cloud Deployment, Capstone

PHASE 1 — FOUNDATIONS: Core Java & Developer Tooling

Day-1 — Introduction to Java & Language Fundamentals

📅 23-Feb-26 | 🕐 1:00 PM – 5:00 PM | *Phase 1: Core Java*

Topics Covered

- Introduction to Java, JVM, JDK, JRE
- Primitive Data Types & Operators
- Flow Control: if/else, switch, loops
- Arrays (1D, varargs, Arrays class)
- Best Practices

Lab Manuals

Lab 1.1: Hello World & JVM Exploration

1. Install JDK 21 and verify with java -version and javac -version
2. Write your first Java program: HelloWorld.java with public static void main
3. Compile and run via command line
4. Explore JVM options: -Xmx, -Xms; observe process with jps

Lab 1.2: Primitive Data Types & Operators

5. Declare variables of all 8 primitive types (byte, short, int, long, float, double, char, boolean)
6. Perform arithmetic, relational, and logical operations
7. Demonstrate integer overflow and type promotion
8. Compare String with == vs .equals()

Lab 1.3: Flow Control – Conditions & Loops

9. Implement FizzBuzz using if/else and switch-case
10. Print multiplication tables using nested for loops
11. Implement a number guessing game using while loop
12. Use break and continue with labeled loops

Lab 1.4: Arrays Deep Dive

13. Create 1D and 2D arrays; use Arrays.sort() and Arrays.binarySearch()
14. Find the largest and second-largest element without sorting
15. Rotate an array to the right by k positions
16. Use varargs method to sum any number of integers

🏠 **Home Assignment:** Write a Java program using all primitive types, loops, and conditionals. Push to GitHub.

Day-2 — Object-Oriented Programming – Classes, Inheritance & Polymorphism

📅 24-Feb-26 | 🕐 1:00 PM – 5:00 PM | *Phase 1: Core Java*

Topics Covered

- Classes, Objects, Constructors, this reference
- Access Specifiers, static keyword
- Wrapper Classes, Type casting, String Handling
- Inheritance, super keyword
- Method Overloading & Overriding
- Abstract Classes & Interfaces
- Runtime Polymorphism
- Best Practices

Lab Manuals

Lab 2.1: Classes & Object Construction

17. Create a BankAccount class with private fields, constructors, getters/setters
18. Add a copy constructor and toString() method
19. Demonstrate the this keyword in constructor chaining
20. Write a static factory method getInstance()

Lab 2.2: Inheritance & super Keyword

21. Create Animal → Dog → Labrador hierarchy
22. Use super() to invoke parent constructors
23. Override toString() at each level
24. Demonstrate final class and final method restrictions

Lab 2.3: Abstract Classes & Interfaces

25. Define abstract Shape class with abstract double area()
26. Implement Circle, Rectangle, Triangle extending Shape
27. Create Drawable and Resizable interfaces with default methods
28. Show a class implementing multiple interfaces

Lab 2.4: Runtime Polymorphism & String Handling

29. Store different Shape subtypes in a Shape[] array; iterate and call area()
30. Demonstrate instanceof and safe casting
31. Build a StringBuilder-based string formatter
32. Benchmark String concatenation vs StringBuilder for 10,000 iterations

🏠 **Home Assignment:** Build a mini OOP model (e.g., Shape hierarchy). Commit & push to Git.

Day-3 — Exception Handling, Collections & Generics

📅 25-Feb-26 | 🕐 1:00 PM – 5:00 PM | *Phase 1: Core Java*

Topics Covered

- Exception Types, Hierarchy, try-catch-finally
- try-with-resources, multi-catch, throw/throws
- User-defined Exceptions
- Collections Framework: List, Set, Map, Queue
- Comparable & Comparator
- Generics with Collections
- Property Files (.properties)

Lab Manuals

Lab 3.1: Exception Handling

33. Catch specific exceptions: ArithmeticException, NumberFormatException, NullPointerException
34. Implement multi-catch and finally block with resource cleanup
35. Create InsufficientFundsException extending RuntimeException
36. Use try-with-resources with a custom AutoCloseable resource

Lab 3.2: Collections – List, Set, Map

37. Compare ArrayList vs LinkedList performance for add/remove operations
38. Use HashSet to remove duplicates from a list; use TreeSet for sorted order
39. Implement a phone book using HashMap; demonstrate getOrDefault and putIfAbsent
40. Simulate a ticket queue with ArrayDeque

Lab 3.3: Sorting with Comparable & Comparator

41. Make a Student class implement Comparable by GPA
42. Use Collections.sort() to sort a list of Students
43. Write a Comparator to sort Students by name, then by GPA descending
44. Chain comparators using thenComparing()

Lab 3.4: Generics & Property Files

45. Write a generic Pair<A,B> class and a generic Stack<T>
46. Build a utility method that prints any List<T>
47. Load app configuration from config.properties using Properties class
48. Demonstrate bounded type parameters with <T extends Number>

🏠 **Home Assignment:** Implement a student grade manager using Collections. Handle exceptions gracefully. Push to Git.

Day-4 — Git & GitHub — Version Control for Professional Developers

📅 26-Feb-26 | 🕐 1:00 PM – 5:00 PM | *Phase 1: Developer Tooling*

Topics Covered

- What is VCS? Why Git?
- Git Installation & Configuration
- Local Repo: init, add, commit, log, diff
- Branching: create, switch, merge, rebase
- Remote Repos: clone, push, pull, fetch
- Resolving Merge Conflicts
- GitHub: Pull Requests, Code Review workflow
- Git Best Practices

Lab Manuals

Lab 4.1: Local Repository Workflow

49. Initialize a local Git repo with git init
50. Stage files with git add, commit with meaningful messages
51. View history with git log --oneline --graph
52. Use git diff to inspect unstaged changes

Lab 4.2: Branching & Merging

53. Create a feature branch: git checkout -b feature/add-login
54. Make commits on both main and feature branch to simulate divergence
55. Merge the feature branch back to main
56. Use git rebase to linearize history

Lab 4.3: Remote Repositories & Conflict Resolution

57. Push repo to GitHub; clone it in a different folder to simulate second developer
58. Pull changes from remote; simulate a merge conflict in the same file
59. Resolve the conflict manually and complete the merge
60. Use git stash to save work-in-progress before switching branches

Lab 4.4: GitHub Pull Request Workflow

61. Fork a peer's repository on GitHub
62. Create a feature branch, make changes, push to your fork
63. Open a Pull Request with a descriptive title and body
64. Review a peer's PR: leave inline comments, request changes, approve and merge

🏠 **Home Assignment:** Collaborate: create a shared GitHub repo, raise a PR, review a peer's code, and merge it.

Day-5 — JUnit 5 — Test-Driven Development

📅 27-Feb-26 | 🕐 1:00 PM – 5:00 PM | *Phase 1: Developer Tooling*

Topics Covered

- Why Testing? Introduction to TDD
- JUnit 5 Setup & Annotations (@Test, @BeforeEach, etc.)
- Writing meaningful assertions
- Test Suites & Fixtures
- Parameterized Tests
- Mocking with Mockito (basics)
- Testing Best Practices & Code Coverage

Lab Manuals

Lab 5.1: JUnit 5 Setup & First Tests

65. Add JUnit 5 dependency in pom.xml
66. Write @Test methods for a Calculator class (add, subtract, multiply, divide)
67. Use assertEquals, assertThrows, assertTrue, assertNull
68. Run tests in IntelliJ and observe green/red results

Lab 5.2: Test Fixtures & Lifecycle

69. Use @BeforeEach to initialize a new object before every test
70. Use @AfterAll to close shared resources after all tests
71. Group related tests using @Nested inner classes
72. Use @DisplayName for human-readable test names

Lab 5.3: Parameterized Tests

73. Use @ParameterizedTest with @ValueSource to test multiple inputs
74. Use @CsvSource to test methods with multiple parameters
75. Use @MethodSource with a static Stream to provide complex test data
76. Assert that invalid inputs throw the correct exception for each case

Lab 5.4: Mocking with Mockito

77. Add Mockito dependency to pom.xml
78. Mock a UserRepository dependency in a UserService test
79. Use when().thenReturn() to stub method responses
80. Verify that mocked methods were called with verify()

🏠 **Home Assignment:** Write unit tests for your Collections exercise (Day-3). Achieve ≥ 80% coverage. Push tests to GitHub.

MILESTONE 1 ✅: Core Java + Git + TDD — Mini code review session. Each participant presents their GitHub repo.

PHASE 2 — DATABASE: SQL, PL/SQL & MySQL

Day-6 — Relational Model & Core SQL

📅 02-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 2: SQL*

Topics Covered

- The Relational Model & RDBMS concepts
- SQL Basics: SELECT, INSERT, UPDATE, DELETE
- DISTINCT, Boolean operators: AND, OR, BETWEEN, LIKE, IN
- ORDER BY, GROUP BY, HAVING
- Set Functions: COUNT, SUM, AVG, MIN, MAX

Lab Manuals

Lab 6.1: MySQL Setup & First Database

81. Install MySQL 8 and connect via MySQL Workbench
82. CREATE DATABASE bookstore; USE bookstore;
83. CREATE TABLE authors (id, name, country, birth_year)
84. INSERT 10 author records; SELECT all and verify

Lab 6.2: Filtering & Sorting

85. SELECT authors from a specific country using WHERE
86. Find books published BETWEEN 2000 AND 2023
87. Use LIKE '%Java%' to search for title substrings
88. Order results by price DESC and title ASC

Lab 6.3: Aggregation & Grouping

89. COUNT total books per genre using GROUP BY
90. Find the average price of books by each publisher
91. Use HAVING to show only publishers with more than 5 books
92. Combine WHERE and HAVING in a single query

Lab 6.4: DML – Insert, Update, Delete

93. Insert a batch of books with a single INSERT statement
94. Update the price of all books in a specific category by 10%
95. Soft-delete a book using an is_active flag rather than DELETE
96. Use TRUNCATE vs DELETE and understand the difference

🏠 **Home Assignment:** Create a MySQL database for a bookstore. Write SELECT queries with filters, grouping, and aggregations.

Day-7 — Advanced SQL — JOINs, DDL & Constraints

📅 03-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 2: SQL*

Topics Covered

- CROSS, INNER, LEFT, RIGHT, FULL OUTER JOINs
- SELF JOIN
- CREATE DATABASE / TABLE, ALTER TABLE, DROP TABLE
- Primary Key, Foreign Key, Constraints, NULL handling
- Indexes: what they are and why they matter
- Subqueries & CTEs (WITH clause)

Lab Manuals

Lab 7.1: JOINs in Practice

97. INNER JOIN books with authors to see books and their author names
98. LEFT JOIN to find all authors, including those with no published books
99. SELF JOIN employees table to find manager-employee pairs
100. Three-table JOIN: orders → books → customers in one query

Lab 7.2: DDL & Constraints

101. ALTER TABLE to add a created_at TIMESTAMP column with DEFAULT NOW()
102. Add a FOREIGN KEY from orders.book_id to books.id
103. Add a UNIQUE constraint on customers.email
104. Use CHECK constraint to ensure price > 0

Lab 7.3: Indexes & Query Optimization

105. Run EXPLAIN SELECT on an un-indexed query; observe full table scan
106. CREATE INDEX on books.title; re-run EXPLAIN and compare
107. Create a composite index on (category, price)
108. Understand when NOT to add indexes (small tables, high-write columns)

Lab 7.4: Subqueries & CTEs

109. Find customers who placed more orders than the average customer (correlated subquery)
110. List all books priced above the average price using a scalar subquery
111. Rewrite the above using a CTE (WITH clause) for readability
112. Use a recursive CTE to generate a hierarchy of categories

🏠 **Home Assignment:** Add relationships to bookstore DB (Authors, Books, Orders). Write JOIN queries. Script everything in Git.

Day-8 — Advanced Querying & Performance Basics

📅 04-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 2: SQL*

Topics Covered

- Window Functions: ROW_NUMBER, RANK, LEAD, LAG
- Advanced Filtering: EXISTS, NOT EXISTS
- Views & Stored Procedures basics
- Transactions: COMMIT, ROLLBACK, SAVEPOINT
- Query optimization basics, EXPLAIN plan

Lab Manuals

Lab 8.1: Window Functions

113. Use ROW_NUMBER() to rank books by price within each category
114. Use RANK() and DENSE_RANK() and compare differences on tied values
115. Use LAG() to compare each order's total to the previous order
116. Use SUM() OVER (PARTITION BY) for running totals per customer

Lab 8.2: Views

117. CREATE VIEW customer_order_summary as a query joining customers and orders
118. SELECT from the view just like a table
119. UPDATE an updatable simple view and verify the underlying table changes
120. DROP VIEW and understand when to use views vs CTEs

Lab 8.3: Stored Procedures & Transactions

121. CREATE PROCEDURE place_order(customer_id, book_id, qty) to insert an order
122. Add transaction control: BEGIN, COMMIT, ROLLBACK inside the procedure
123. Use SAVEPOINT to partially rollback only part of a multi-step transaction
124. Call the procedure and verify atomicity by simulating a failure midway

Lab 8.4: Query Performance Analysis

125. Use EXPLAIN FORMAT=JSON on a complex JOIN query
126. Identify and resolve a missing index causing a full scan
127. Rewrite a slow correlated subquery using a JOIN for better performance
128. Use SHOW PROCESSLIST and KILL to manage slow queries

🏠 **Home Assignment:** Write a stored procedure for bookstore order processing. Use transactions.

Day-9 — PL/SQL Fundamentals

📅 05-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 2: PL/SQL*

Topics Covered

- What is PL/SQL? Architecture & Benefits
- PL/SQL Block Structure: Declaration, Execution, Exception
- Variables, Data Types, Constants
- Conditional Control: IF-THEN-ELSE, CASE
- Looping: LOOP, WHILE, FOR
- NULL handling: NULLIF, COALESCE
- Exception Handling in PL/SQL
- Cursors (implicit & explicit)

Lab Manuals

Lab 9.1: PL/SQL Block Structure

129. Write a basic DECLARE/BEGIN/EXCEPTION/END block to print Hello from PL/SQL
130. Declare variables of different types (VARCHAR2, NUMBER, DATE, BOOLEAN)
131. Use %TYPE and %ROWTYPE to anchor variable types to table columns
132. Use DBMS_OUTPUT.PUT_LINE to print results

Lab 9.2: Conditional Logic & Loops

133. Calculate a student's grade (A/B/C/F) using IF-ELSIF-ELSE
134. Rewrite using a CASE expression
135. Print the first 10 Fibonacci numbers using a LOOP with EXIT WHEN
136. Iterate over a number range using FOR i IN 1..10 LOOP

Lab 9.3: Cursors – Implicit & Explicit

137. Use an implicit cursor with SQL%ROWCOUNT to count affected rows in UPDATE
138. Declare an explicit cursor to fetch all books with price > 500
139. Use cursor FOR LOOP (simplest cursor iteration pattern)
140. Fetch one row at a time with FETCH…INTO and close cursor manually

Lab 9.4: Exception Handling in PL/SQL

141. Handle NO_DATA_FOUND and TOO_MANY_ROWS system exceptions
142. Create a user-defined exception for invalid_discount
143. Use SQLCODE and SQLERRM in the WHEN OTHERS handler
144. Write a procedure that logs errors to an error_log table

🏠 **Home Assignment:** Write PL/SQL blocks to compute student grade summaries with error handling. Commit SQL scripts to Git.

MILESTONE 2 ✅: SQL + PL/SQL Proficiency — SQL quiz + hands-on: JOIN queries, aggregations, and a stored procedure.

PHASE 3 — BACKEND: Hibernate, Spring Boot & REST APIs

Day-10 — ORM Concepts & JPA Introduction

📅 06-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 3: Hibernate / JPA*

Topics Covered

- Data Persistence & ORM overview
- JPA Specification & Hibernate as implementation
- Entity Classes: requirements, annotations (@Entity, @Table)
- Persistent Fields & Properties
- Primary Keys: @Id, @GeneratedValue strategies
- Validating Persistent Fields (@NotNull, @Size, etc.)
- Setting up Hibernate with Maven

Lab Manuals

Lab 10.1: Maven Project Setup with Hibernate

145. Create a Maven project; add hibernate-core and mysql-connector-java dependencies
146. Configure persistence.xml with MySQL connection details
147. Set hibernate.hbm2ddl.auto=create to let Hibernate generate schema
148. Verify Hibernate connects to MySQL and creates an empty schema

Lab 10.2: Your First JPA Entity

149. Create a Product entity class annotated with @Entity and @Table
150. Map fields: id (@Id, @GeneratedValue), name, price, category
151. Use @Column(nullable=false, length=100) for constraints
152. Persist a Product using EntityManager.persist() and verify in DB

Lab 10.3: Bean Validation on Entities

153. Add hibernate-validator dependency to pom.xml
154. Annotate id with @NotNull, name with @Size(min=2, max=100), price with @Min(0)
155. Write a test that attempts to persist an invalid entity and catches ConstraintViolationException
156. Use a Validator to validate objects before persisting

Lab 10.4: CRUD Operations with EntityManager

157. Persist three products in one transaction; verify with a SELECT in Workbench
158. Find a product by ID using EntityManager.find()
159. Update a product by merging a detached entity
160. Remove a product and confirm deletion in the database

🏠 **Home Assignment:** Map a 'Product' entity to a MySQL table using JPA. Persist and retrieve data.

Day-11 — Entity Lifecycle, Relationships & Querying

📅 09-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 3: Hibernate / JPA*

Topics Covered

- EntityManager Interface & Lifecycle
- Persisting, Finding, Removing, Merging entities
- Entity Relationships: @OneToOne, @OneToMany, @ManyToMany
- Bidirectional vs Unidirectional
- Cascade Operations & Orphan Removal
- JPQL & Criteria API

Lab Manuals

Lab 11.1: Entity Lifecycle States

161. Demonstrate the four lifecycle states: New, Managed, Detached, Removed
162. Use EntityManager.contains() and EntityManager.detach() to inspect state
163. Show that changes to a Managed entity are auto-flushed at transaction commit
164. Use EntityManager.refresh() to reload entity from DB

Lab 11.2: @OneToMany & @ManyToOne Relationships

165. Create Order entity with a @ManyToOne to Customer
166. Add @OneToMany(mappedBy) on Customer.orders for the inverse side
167. Persist a Customer with 3 Orders in one transaction using CascadeType.PERSIST
168. Fetch orders for a customer and print them

Lab 11.3: @ManyToMany & Orphan Removal

169. Create Student and Course entities with a @ManyToMany relationship
170. Use a join table enrollment(student_id, course_id)
171. Enroll a student in two courses; unenroll by removing from the collection
172. Enable orphanRemoval=true and verify dependent entities are deleted

Lab 11.4: JPQL Queries

173. Write a JPQL SELECT q FROM Product p WHERE p.price > :minPrice
174. Use a named query @NamedQuery on the entity class
175. Write an aggregate JPQL query to find average price per category
176. Convert a JPQL query to a Criteria API query for type safety

🏠 **Home Assignment:** Model an e-commerce schema (User, Product, Order, OrderItem) with JPA relationships. Run JPQL queries.

Day-12 — Spring Core — DI, IoC & Spring MVC

📅 10-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 3: Spring Framework*

Topics Covered

- Spring Framework overview & ecosystem
- Dependency Injection (DI) & Inversion of Control (IoC)
- Spring Beans: XML config vs annotations
- Bean Scopes & Lifecycle
- Aspect-Oriented Programming (AOP) basics
- Spring MVC architecture & DispatcherServlet
- Configuring Controllers & Request Mappings

Lab Manuals

Lab 12.1: Dependency Injection with @Autowired

177. Create a Spring project with spring-context dependency
178. Define a NotificationService interface with EmailNotification and SMSNotification implementations
179. Inject NotificationService into OrderService using @Autowired on constructor
180. Swap the implementation by changing @Primary annotation; no OrderService code changes needed

Lab 12.2: Bean Scopes & Lifecycle

181. Create a Singleton bean and a Prototype bean; compare instance identities
182. Implement @PostConstruct and @PreDestroy lifecycle callbacks
183. Use @Scope("request") in a web context (requires Spring Web)
184. Register a BeanFactoryPostProcessor to modify bean definitions at startup

Lab 12.3: Spring AOP – Logging & Timing

185. Add spring-aspects dependency; enable @EnableAspectJAutoProxy
186. Create a LoggingAspect with @Before advice to log method entry
187. Add @AfterReturning to log the return value
188. Add @Around advice to measure method execution time in milliseconds

Lab 12.4: Spring MVC – Controller & Views

189. Create a Spring MVC project with spring-webmvc and a web.xml configuring DispatcherServlet
190. Write a @Controller with @GetMapping(/products) returning a model+view
191. Configure InternalResourceViewResolver to resolve JSP views
192. Submit a form with @PostMapping and redirect using "redirect:/products"

🏠 **Home Assignment:** Create a simple Spring MVC app that handles a form submission. Wire beans using annotations.

Day-13 — Spring Boot — Getting Started & Auto-Configuration

📅 11-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 3: Spring Boot*

Topics Covered

- Why Spring Boot? Convention over Configuration
- Spring Initializr, Starters, CLI
- @SpringBootApplication, Application class
- Auto-configuration, Component Scans
- application.properties / application.yml
- Embedded Servers (Tomcat, Jetty)
- Logging with Spring Boot
- Profiles (dev, prod)
- Build Systems: Maven vs Gradle

Lab Manuals

Lab 13.1: Bootstrap a Spring Boot Project

193. Generate a project at start.spring.io with Web, JPA, MySQL starters
194. Review the generated pom.xml and @SpringBootApplication class
195. Run the app with mvn spring-boot:run; confirm Tomcat starts on port 8080
196. Change the port to 8081 in application.properties and restart

Lab 13.2: Auto-Configuration & Component Scan

197. Add @Service and @Repository beans; verify they are discovered automatically
198. Use @ComponentScan to specify a custom base package
199. Print all registered beans using ApplicationContext.getBeanDefinitionNames()
200. Exclude an auto-configuration class using @SpringBootApplication(exclude=...)

Lab 13.3: Profiles & External Configuration

201. Create application-dev.properties and application-prod.properties with different DB URLs
202. Switch active profile with --spring.profiles.active=dev at runtime
203. Use @Value to inject properties into a bean
204. Use @ConfigurationProperties to bind a group of properties to a POJO

Lab 13.4: Logging Configuration

205. Change root log level to DEBUG in application.properties
206. Set a custom log level for your package only
207. Add a logback-spring.xml to configure rolling file appender
208. Use @Slf4j (Lombok) or LoggerFactory to add structured log messages

🏠 **Home Assignment:** Bootstrap a Spring Boot project. Configure profiles for dev and prod. Run with embedded Tomcat.

Day-14 — Building RESTful APIs with Spring Boot

📅 12-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 3: Spring Boot*

Topics Covered

- REST principles: Resources, HTTP Verbs, Status Codes
- @RestController, @RequestMapping, @GetMapping, etc.
- Request/Response: @RequestBody, @ResponseBody, @PathVariable, @RequestParam
- Jackson: JSON serialization/deserialization
- DTOs & Mappers
- Exception Handling: @ControllerAdvice, @ExceptionHandler
- Returning proper HTTP status codes
- API versioning basics

Lab Manuals

Lab 14.1: Build a Product REST API

209. Create ProductController with @RestController and /api/v1/products base path
210. Implement GET /products (list all), GET /products/{id} (get one)
211. Implement POST /products (create) with @RequestBody ProductDTO
212. Implement PUT /products/{id} (update) and DELETE /products/{id}

Lab 14.2: Request Parameters & Path Variables

213. Add @RequestParam for filtering: GET /products?category=Electronics&minPrice=100
214. Add @PathVariable for resource identifier: GET /products/{id}
215. Return ResponseEntity<ProductDTO> with appropriate status codes (200, 201, 404)
216. Test all endpoints with Postman; verify correct HTTP status codes

Lab 14.3: DTOs & Jackson Customization

217. Create ProductRequestDTO and ProductResponseDTO to separate API model from entity
218. Use @JsonProperty to rename a field in JSON output
219. Use @JsonIgnore to hide sensitive fields like internalCode
220. Configure Jackson globally to use snake_case naming strategy

Lab 14.4: Global Exception Handling

221. Create a GlobalExceptionHandler class annotated with @ControllerAdvice
222. Handle ResourceNotFoundException with a 404 response and error body
223. Handle MethodArgumentNotValidException (validation errors) with 400 and field errors
224. Handle all other exceptions with a generic 500 handler; never expose stack traces to clients

🏠 **Home Assignment:** Build a complete CRUD REST API for 'Products' with proper error handling and status codes. Test with Postman.

Day-15 — Spring Data JPA — Repositories & Query Methods

📅 13-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 3: Spring Data JPA*

Topics Covered

- Spring Data JPA Overview
- JpaRepository, CrudRepository interfaces
- Defining Repository Interfaces
- Query Methods: findBy, existsBy, countBy
- @Query annotation (JPQL & native)
- Named Queries
- Pagination & Sorting (Pageable)
- Transactions with @Transactional
- Auditing (@CreatedDate, @LastModifiedDate)

Lab Manuals

Lab 15.1: JpaRepository Setup

225. Create ProductRepository extends JpaRepository<Product, Long>
226. Use productRepository.save(), findById(), findAll(), deleteById() in a service
227. Confirm Spring Data generates the implementation at runtime
228. Inspect the auto-generated SQL with spring.jpa.show-sql=true

Lab 15.2: Derived Query Methods

229. Write findByCategory(String category) – Spring generates the SQL automatically
230. Write findByCategoryAndPriceLessThan(String cat, BigDecimal price)
231. Write existsByName(String name) and countByCategory(String category)
232. Use Optional<Product> as return type for single-result finders

Lab 15.3: Pagination & Sorting

233. Add Pageable parameter to findAll(Pageable pageable) in service method
234. Call the endpoint with ?page=0&size=5&sort=price,desc
235. Return Page<ProductDTO> to include total pages and total elements in response
236. Implement keyset pagination for high-performance large datasets

Lab 15.4: Auditing & @Transactional

237. Enable JPA auditing with @EnableJpaAuditing on the main class
238. Add @CreatedDate and @LastModifiedDate fields to a base entity class
239. Use @EntityListeners(AuditingEntityListener.class) on the entity
240. Demonstrate @Transactional(readOnly=true) on service read methods for performance

🏠 **Home Assignment:** Integrate Spring Data JPA with your REST API (Day-14). Add pagination and sorting to GET endpoints.

Day-16 — Spring Data REST, Validation & API Documentation

📅 16-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 3: Spring Boot*

Topics Covered

- Spring Data REST: auto-exposing repositories as REST
- HAL, HATEOAS concepts
- Bean Validation: @Valid, @NotNull, @Size, custom validators
- Handling validation errors gracefully
- Swagger / OpenAPI with springdoc-openapi
- Documenting controllers and DTOs
- Actuator: health, metrics, info endpoints

Lab Manuals

Lab 16.1: Bean Validation on DTOs

241. Annotate ProductRequestDTO with @NotBlank, @Min, @Size on relevant fields
242. Add @Valid in the controller method parameter to trigger validation
243. Return a structured error response with field name and message from @ControllerAdvice
244. Write a custom @PriceRange validator as a custom constraint annotation

Lab 16.2: Swagger / OpenAPI Documentation

245. Add springdoc-openapi-starter-webmvc-ui dependency
246. Access http://localhost:8080/swagger-ui.html and explore auto-generated docs
247. Add @Operation and @Parameter annotations to improve endpoint descriptions
248. Add @Schema annotations on DTO classes to document fields

Lab 16.3: Spring Actuator

249. Add spring-boot-starter-actuator dependency
250. Access /actuator/health, /actuator/info, /actuator/metrics
251. Expose all actuator endpoints in application.properties (management.endpoints.web.exposure.include=*)
252. Create a custom health indicator that checks database connectivity

Lab 16.4: Spring Data REST Exploration

253. Add spring-boot-starter-data-rest; observe auto-generated REST endpoints
254. Access the HAL browser to navigate hypermedia links
255. Customize exported resource path with @RepositoryRestResource(path=...)
256. Hide a repository method from REST exposure using @RestResource(exported=false)

🏠 **Home Assignment:** Add validation to your API. Generate Swagger UI documentation. Expose Actuator endpoints.

Day-17 — Spring Security — Authentication & Authorization

📅 17-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 3: Spring Boot*

Topics Covered

- Why Security? Common vulnerabilities (OWASP Top 10 overview)
- Spring Security architecture
- HTTP Basic Auth & Form Login
- JWT (JSON Web Tokens): structure, signing, verification
- Securing REST APIs with JWT
- Role-based Access Control (@PreAuthorize)
- CORS configuration for React frontend
- Storing passwords securely (BCrypt)

Lab Manuals

Lab 17.1: Spring Security Basics

257. Add spring-boot-starter-security; observe all endpoints now require Basic Auth
258. Create a UserDetailsService loading users from the database
259. Configure SecurityFilterChain with HttpSecurity to permit /auth/** and secure /api/**
260. Test secured endpoints with Postman using Authorization: Basic header

Lab 17.2: JWT Token Generation & Validation

261. Add io.jsonwebtoken (jjwt) dependency for JWT support
262. Create JwtUtil class with generateToken(username) and validateToken(token) methods
263. Implement /api/auth/login endpoint that returns a JWT on valid credentials
264. Write a JwtRequestFilter extending OncePerRequestFilter to validate JWT on every request

Lab 17.3: Role-Based Access Control

265. Create ROLE_USER and ROLE_ADMIN in the users table
266. Enable @EnableMethodSecurity on the security config class
267. Add @PreAuthorize("hasRole('ADMIN')") on DELETE /api/products/{id}
268. Test that USER role cannot delete; ADMIN can. Verify with different JWT tokens in Postman

Lab 17.4: CORS & Password Encoding

269. Configure CORS in SecurityFilterChain to allow http://localhost:3000 (React dev server)
270. Register BCryptPasswordEncoder as a Spring bean
271. Encode passwords before storing in DB; verify in login that encoder.matches() works
272. Test full login flow: POST /auth/login → receive JWT → call protected endpoint with Bearer token

🏠 **Home Assignment:** Secure your REST API with JWT. Implement login endpoint returning token. Protect routes by role.

MILESTONE 3 ✅: Full Backend API — Participants demo their REST API: CRUD operations, validation, JWT security, Swagger docs.

PHASE 4 — FRONTEND: JavaScript, React.js & API Integration

Day-18 — Modern JavaScript (ES6+) Fundamentals

📅 18-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 4: JavaScript*

Topics Covered

- JavaScript in the browser: DOM, events
- Variables: var, let, const — scoping differences
- Arrow functions, Template literals, Destructuring
- Spread/Rest operators
- Array methods: map, filter, reduce, find
- Promises, async/await
- ES6 Modules: import/export
- Fetch API & HTTP requests
- JSON: parsing & stringifying

Lab Manuals

Lab 18.1: ES6 Syntax & Scoping

273. Compare var hoisting issues vs block-scoped let and const with concrete examples
274. Write arrow functions and compare this binding with regular functions
275. Use template literals to build HTML strings with embedded expressions
276. Use destructuring on objects and arrays; use rest and spread with function arguments

Lab 18.2: Array Methods Deep Dive

277. Use .map() to transform an array of products into display strings
278. Use .filter() to show only products in stock
279. Use .reduce() to calculate the total cart value
280. Chain .filter().map().sort() in one expression

Lab 18.3: Promises & async/await

281. Create a promise that resolves after 1 second using setTimeout
282. Chain .then().catch().finally() on the promise
283. Rewrite the same logic using async/await; handle errors with try/catch
284. Use Promise.all() to run two fetch calls in parallel and wait for both

Lab 18.4: Fetch API & DOM Manipulation

285. Fetch data from https://jsonplaceholder.typicode.com/posts using fetch()
286. Parse the JSON response and render 10 post titles into a <ul> in the DOM
287. Add a click event listener to each item to show the post body in a modal
288. Use AbortController to cancel a slow fetch request

🏠 **Home Assignment:** Build a JavaScript mini-app that fetches data from a public API and renders it in the DOM.

Day-19 — Ajax, REST Consumption & Node.js / npm Basics

📅 19-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 4: JavaScript*

Topics Covered

- Ajax: XMLHttpRequest vs Fetch vs Axios
- RESTful Web Services consumption
- HTTP Verbs, Status Codes, Headers
- JSON & XML data formats
- npm: package.json, node_modules
- Node.js core modules: fs, path, http
- npm scripts (build, start, test)
- Introduction to Vite

Lab Manuals

Lab 19.1: Axios vs Fetch

289. Compare XMLHttpRequest, fetch(), and Axios for a GET request to JSONPlaceholder
290. Use Axios to POST a new resource; log the created object returned by the server
291. Add an Axios request interceptor to attach an Authorization header to every request
292. Handle Axios errors in a catch block; distinguish network errors from HTTP error status codes

Lab 19.2: Consuming Your Spring Boot API

293. Start the Spring Boot API from Day 17 (or use a running instance)
294. Use Axios to call GET /api/products from a plain HTML page
295. Handle the CORS preflight by verifying your Spring Boot CORS configuration
296. Implement a simple search form that calls GET /api/products?category=X on submit

Lab 19.3: Node.js & npm Basics

297. Initialize a new npm project with npm init -y; review package.json
298. Install lodash and axios; observe node_modules and package-lock.json
299. Write a Node.js script using fs to read a JSON file and parse it
300. Add custom scripts in package.json: start, build, lint

Lab 19.4: Vite Setup

301. Scaffold a Vanilla JS project: npm create vite@latest my-app -- --template vanilla
302. Run npm install and npm run dev; observe HMR (Hot Module Replacement)
303. Create two JS modules: utils.js and main.js using ES6 import/export
304. Build for production with npm run build and inspect the dist folder

🏠 **Home Assignment:** Use Axios to call your Spring Boot API from a plain HTML/JS page. Understand CORS. Setup a Node project.

Day-20 — React Fundamentals — Components, JSX & Props

📅 20-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 4: React.js*

Topics Covered

- What is React? Virtual DOM, ecosystem
- Setting up with Vite
- JSX syntax & rules
- Functional Components
- Props: passing, destructuring, PropTypes
- Rendering: conditional rendering, lists with map()
- Keys in lists
- React DevTools

Lab Manuals

Lab 20.1: First React App with Vite

305. Scaffold React project: npm create vite@latest shop-app -- --template react
306. Delete boilerplate; create a clean App.jsx with a Hello React heading
307. Run npm run dev and open React DevTools in Chrome to inspect the component tree
308. Add a <header> with app name and a <main> content section

Lab 20.2: Functional Components & JSX

309. Create a ProductCard component accepting name, price, category as props
310. Use JSX expressions {} to render dynamic values and a ternary for in-stock badge
311. Create a ProductList component that accepts an array and maps over it to render ProductCards
312. Pass a products array as prop from App.jsx to ProductList

Lab 20.3: Props & PropTypes

313. Add PropTypes validation to ProductCard: name (string, required), price (number, required)
314. Test what happens when a required prop is missing (console warning)
315. Use defaultProps to set fallback values for optional props
316. Destructure props directly in the function signature for cleaner code

Lab 20.4: Conditional Rendering & Keys

317. Conditionally render a 'No products found' message when the list is empty
318. Use {condition && <Component />} syntax for showing/hiding a badge
319. Add a unique key={product.id} to each mapped item; remove it and observe the console warning
320. Implement a simple filter by category using array .filter() before .map()

🏠 **Home Assignment:** Create a React app that displays a list of products fetched from your Spring Boot API. Use props to pass data.

Day-21 — State Management — Hooks & Component Communication

📅 23-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 4: React.js*

Topics Covered

- useState: local state management
- useEffect: side effects, data fetching, cleanup
- useRef: DOM refs and persisting values
- Event handling in React
- Lifting state up
- Component lifecycle with hooks
- Custom hooks
- Context API: createContext, useContext

Lab Manuals

Lab 21.1: useState & Event Handling

321. Add a counter component with useState; increment, decrement, and reset buttons
322. Build a toggle to show/hide product details on click
323. Create a text search input with useState; filter product list in real-time
324. Understand that state updates are asynchronous; use functional update form: setCount(prev => prev + 1)

Lab 21.2: useEffect & Data Fetching

325. Fetch products from Spring Boot API inside useEffect with an empty dependency array []
326. Show a loading spinner while fetching; hide it when data arrives
327. Add error state to display a user-friendly message if fetch fails
328. Implement cleanup: cancel fetch on component unmount using AbortController

Lab 21.3: Lifting State & useRef

329. Move cart state from CartIcon up to App.jsx so multiple components can read it
330. Pass cart state and an addToCart function down to ProductCard and CartPage as props
331. Use useRef to focus the search input automatically when the page loads
332. Use useRef to store a previous value without triggering a re-render

Lab 21.4: Context API for Global State

333. Create CartContext with createContext(); provide it at the App level with CartContext.Provider
334. Consume cart state in CartIcon and CartPage using useContext(CartContext)
335. Add addToCart and removeFromCart functions to the context value
336. Create a custom hook useCart() that wraps useContext for a cleaner API

🏠 **Home Assignment:** Add a shopping cart feature using useState and Context API. Cart state shared across components.

Day-22 — Forms, Validation & React Router

📅 24-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 4: React.js*

Topics Covered

- Controlled vs Uncontrolled components
- Handling form inputs, textarea, select
- Form validation (manual & with React Hook Form)
- Displaying validation errors
- React Router v6: BrowserRouter, Routes, Route
- Nested routes & layouts
- Navigation: useNavigate, Link, NavLink
- Protected routes (auth guard)

Lab Manuals

Lab 22.1: Controlled Forms

337. Build a Login form with email and password as controlled inputs using useState
338. Disable the Submit button unless both fields are non-empty
339. Implement manual validation: check email format with regex; show inline error messages
340. Reset the form after successful submission

Lab 22.2: React Hook Form

341. Install react-hook-form; replace manual state with useForm()
342. Use register() on each input and errors object for validation messages
343. Add required, minLength, pattern rules declaratively on register()
344. Use watch() to show a password strength indicator in real-time

Lab 22.3: React Router v6 Setup

345. Wrap App in <BrowserRouter>; define <Routes> with <Route> for /, /products, /cart, /login
346. Add a shared <Layout> component with a Navbar that renders on all pages
347. Use <Link> and <NavLink> in Navbar; observe active class on NavLink
348. Use useNavigate() to programmatically redirect after form submission

Lab 22.4: Protected Routes

349. Create a ProtectedRoute component that checks if a JWT token exists in localStorage
350. If no token, redirect to /login using <Navigate to='/login' replace />
351. Wrap /cart and /orders routes in <ProtectedRoute>
352. Simulate login/logout by setting/clearing the token and verify route protection

🏠 **Home Assignment:** Build Login & Registration forms with validation. Add routing: Home, Products, Cart, Login pages. Guard Cart route.

Day-23 — Advanced State — Redux Toolkit & API Integration

📅 25-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 4: React.js*

Topics Covered

- Why Redux? When to use it vs Context
- Redux Toolkit: createSlice, configureStore
- Actions, Reducers, Selectors
- useSelector & useDispatch hooks
- RTK Query: data fetching & caching
- Integrating with your Spring Boot JWT API
- Storing JWT token, Auth state in Redux
- Axios interceptors for adding token to requests

Lab Manuals

Lab 23.1: Redux Toolkit Store Setup

353. Install @reduxjs/toolkit and react-redux
354. Create authSlice with login and logout reducers; define initial state
355. Configure store with configureStore({ reducer: { auth: authReducer } })
356. Wrap App in <Provider store={store}> in main.jsx

Lab 23.2: Auth Slice & Dispatch

357. Create a loginThunk async thunk that POSTs to /api/auth/login
358. On success, dispatch login(token) action to store the token
359. Use useSelector(state => state.auth.token) in Navbar to show/hide Login button
360. Dispatch logout() on a logout button click and clear the stored token

Lab 23.3: RTK Query for Products API

361. Create a productsApi using createApi with baseQuery pointing to Spring Boot
362. Define getProducts endpoint (GET /api/v1/products with optional params)
363. Add createProduct, updateProduct, deleteProduct mutations
364. Use auto-generated useGetProductsQuery() hook in ProductList; no useEffect needed

Lab 23.4: Axios Interceptor for JWT

365. Create an Axios instance with baseURL pointing to Spring Boot
366. Add a request interceptor to read token from Redux store and attach as Bearer header
367. Add a response interceptor to catch 401 errors and dispatch logout() automatically
368. Test that protected API calls succeed with a valid token and redirect to /login when expired

🏠 **Home Assignment:** Integrate Redux Toolkit into your app. Manage auth state. RTK Query to fetch products. Auto-attach JWT header.

Day-24 — Styling, UI Libraries & Performance

📅 26-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 4: React.js*

Topics Covered

- CSS Modules: scoped component styles
- Tailwind CSS: utility-first approach
- Styled-components: CSS-in-JS
- Responsive design & media queries
- React component libraries: Material UI or Ant Design
- React.memo, useMemo, useCallback
- Lazy loading with React.lazy & Suspense
- Code splitting

Lab Manuals

Lab 24.1: Tailwind CSS Setup & Styling

369. Install Tailwind CSS and configure tailwind.config.js in the Vite project
370. Replace all inline/CSS styles on ProductCard with Tailwind utility classes
371. Build a responsive grid: 1 column on mobile, 3 on desktop using Tailwind breakpoints
372. Add a dark mode toggle using Tailwind dark: variant

Lab 24.2: CSS Modules

373. Rename App.css to App.module.css; import styles as a module object
374. Use styles.container, styles.header class names in JSX
375. Show that CSS Module class names are hashed to prevent collisions
376. Combine module classes with Tailwind classes using clsx or classnames library

Lab 24.3: Performance – Memoization

377. Create a slow-rendering ProductList; use React Profiler in DevTools to measure render time
378. Wrap ProductCard with React.memo() to prevent re-renders when parent state changes
379. Use useMemo() to memoize an expensive calculation (e.g., total cart value)
380. Use useCallback() to stabilize a function reference passed as prop

Lab 24.4: Code Splitting & Lazy Loading

381. Replace static imports of AdminPage and CartPage with React.lazy(() => import(...))
382. Wrap lazy routes in <Suspense fallback={<Spinner />}>
383. Build the app and inspect the dist folder; observe code split chunks
384. Implement a custom loading skeleton component as the Suspense fallback

🏠 **Home Assignment:** Redesign your app UI using Tailwind CSS. Add a loading spinner component. Implement lazy loading for routes.

MILESTONE 4 ✅: Full Frontend — Demo: Login with JWT, view products list, add to cart, navigate routes.

PHASE 5 — INTEGRATION, CI/CD & PRODUCTION DEPLOYMENT

Day-25 — Full-Stack Integration & End-to-End Testing

📅 27-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 5: Integration*

Topics Covered

- Connecting React frontend to Spring Boot backend in production mode
- Environment variables in React (.env)
- CORS deep-dive: configuration in Spring Boot
- API error handling end-to-end
- Postman Automated Tests
- React Testing Library basics
- End-to-end request flow from React → REST API → DB

Lab Manuals

Lab 25.1: Environment Configuration

385. Create .env.development and .env.production with different VITE_API_URL values
386. Access env vars in React using import.meta.env.VITE_API_URL
387. Build the React app for production (npm run build) and serve with a static server
388. Configure the production API URL to point to the deployed Spring Boot instance

Lab 25.2: CORS Deep Dive

389. Observe a CORS preflight error in the browser Network tab when origins mismatch
390. Fix CORS in Spring Boot using @CrossOrigin or a global WebMvcConfigurer
391. Understand the difference between simple requests and preflighted requests
392. Test that OPTIONS preflight returns 200 with correct headers in Postman

Lab 25.3: React Testing Library

393. Install @testing-library/react and @testing-library/jest-dom
394. Write a test that renders ProductCard and asserts the product name is visible in the DOM
395. Fire a click event on the Add to Cart button and assert a callback was called
396. Mock Axios in tests using jest.mock('axios') to avoid real network calls

Lab 25.4: Postman Automated Tests

397. In Postman, write a test script to assert response status is 200 for GET /products
398. Save the JWT token from the login response into a Postman environment variable
399. Use the saved token as a Bearer header in subsequent requests automatically
400. Run the entire collection with Newman (Postman CLI) from the terminal

🏠 **Home Assignment:** Run full stack locally. Write at least 3 integration-style tests. Identify and fix CORS/auth issues.

Day-26 — Jenkins — CI/CD Pipelines

📅 30-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 5: CI/CD*

Topics Covered

- What is CI/CD? Why it matters
- Jenkins Installation & Configuration
- Creating Jenkins Jobs (Freestyle & Pipeline)
- Jenkinsfile: Declarative Pipeline syntax
- Stages: Checkout → Build → Test → Package
- Integrating Jenkins with GitHub (webhooks)
- Running JUnit tests in pipeline
- Archiving artifacts & build reports

Lab Manuals

Lab 26.1: Jenkins Setup

401. Run Jenkins in Docker: docker run -p 8085:8080 jenkins/jenkins:lts
402. Complete the setup wizard; install recommended plugins
403. Create a Freestyle job to pull your Spring Boot project from GitHub and run mvn package
404. View the build console output and confirm success

Lab 26.2: Declarative Pipeline (Jenkinsfile)

405. Create a Jenkinsfile in the root of your Spring Boot repo with stages: Checkout, Build, Test, Package
406. Push the Jenkinsfile to GitHub; configure Jenkins Pipeline to read from SCM
407. Run the pipeline; observe each stage on the Blue Ocean view
408. Add a post block to always archive the JAR file as a build artifact

Lab 26.3: GitHub Webhooks

409. Expose Jenkins to the internet using ngrok (for local setup) or use a cloud Jenkins instance
410. Configure a GitHub webhook: Payload URL = http://your-jenkins/github-webhook/
411. Push a commit to GitHub and watch Jenkins auto-trigger the pipeline within seconds
412. Break a unit test intentionally; push and observe the pipeline fail with the failed test report

Lab 26.4: Test Reports & Notifications

413. Add junit('**/target/surefire-reports/*.xml') to the Jenkinsfile to publish test results
414. View the JUnit test trend graph in Jenkins after 3 builds
415. Add a post { failure { mail ... } } block to send email on build failure
416. Configure build status badge in your GitHub README using the Jenkins badge plugin

🏠 **Home Assignment:** Create a Jenkins pipeline for your Spring Boot app: pulls from GitHub, builds with Maven, runs JUnit, packages JAR.

Day-27 — Docker — Containerizing Your Applications

📅 31-Mar-26 | 🕐 1:00 PM – 5:00 PM | *Phase 5: CI/CD*

Topics Covered

- What is Docker? VMs vs Containers
- Docker Architecture: images, containers, registry
- Dockerfile: writing, layering, best practices
- Docker commands: build, run, stop, ps, logs, exec
- Multi-stage builds for Spring Boot JAR
- Dockerizing the React app (with Nginx)
- Docker Compose: running multi-container apps

Lab Manuals

Lab 27.1: Docker Basics

417. Install Docker Desktop; run hello-world to verify installation
418. Run mysql:8 container with docker run -e MYSQL_ROOT_PASSWORD=pass -p 3306:3306 mysql:8
419. Use docker ps, docker logs, docker exec -it to interact with running containers
420. Pull the openjdk:21-slim image; inspect its layers with docker history

Lab 27.2: Dockerfile for Spring Boot

421. Write a Dockerfile: FROM openjdk:21-slim, COPY target/app.jar, EXPOSE 8080, ENTRYPOINT
422. Build the image: docker build -t my-spring-api:1.0 .
423. Run the container: docker run -p 8080:8080 my-spring-api:1.0
424. Optimize with a multi-stage build: stage 1 uses Maven to build JAR, stage 2 copies only the JAR

Lab 27.3: Dockerfile for React App

425. Write a multi-stage Dockerfile: stage 1 (node:18-alpine) runs npm run build, stage 2 (nginx:alpine) serves dist/
426. Create nginx.conf with try_files $uri /index.html for React Router
427. Build and run the React container on port 3000
428. Test the full frontend works inside Docker including route navigation

Lab 27.4: Docker Compose

429. Create docker-compose.yml with three services: mysql, spring-api, react-app
430. Use depends_on and healthcheck to ensure MySQL is ready before Spring Boot starts
431. Use a named volume for MySQL data persistence across restarts
432. Run docker compose up --build and test the full stack at http://localhost:3000

🏠 **Home Assignment:** Write Dockerfiles for both apps. Create docker-compose.yml. Run entire stack with one command.

Day-28 — Cloud Deployment — AWS / Render / Railway

📅 01-Apr-26 | 🕐 1:00 PM – 5:00 PM | *Phase 5: Deployment*

Topics Covered

- Cloud deployment options overview (AWS EC2, Elastic Beanstalk, Render, Railway)
- Deploying Spring Boot JAR to a cloud service
- Setting up environment variables for production
- Deploying React app to Netlify or Vercel
- Setting up a cloud MySQL database (AWS RDS or PlanetScale)
- Domain & HTTPS basics
- Reading application logs in production
- Health checks & monitoring basics

Lab Manuals

Lab 28.1: Deploy Spring Boot to Render

433. Create a free account on Render.com; click New → Web Service
434. Connect to your GitHub repo; set build command to mvn package -DskipTests and start command to java -jar target/*.jar
435. Add environment variables: SPRING_DATASOURCE_URL, SPRING_DATASOURCE_USERNAME, SPRING_DATASOURCE_PASSWORD
436. Wait for the deploy; open the live URL and hit /actuator/health

Lab 28.2: Cloud MySQL Database

437. Create a free MySQL database on Railway or PlanetScale
438. Copy the connection string and update your Spring Boot environment variables on Render
439. Run your app and verify it connects to the cloud DB (check Hibernate schema creation logs)
440. Insert a test record via the API and verify it appears in the cloud DB dashboard

Lab 28.3: Deploy React App to Vercel

441. Create a free Vercel account; import your React GitHub repository
442. Set VITE_API_URL environment variable in Vercel to your Render backend URL
443. Vercel auto-detects Vite; click Deploy and wait ~1 minute for the live URL
444. Test the full frontend: login, browse products, add to cart – all using the deployed backend

Lab 28.4: Production Monitoring

445. Configure your Spring Boot app to log to stdout (not file) for cloud log capture
446. View live application logs in the Render dashboard
447. Access /actuator/health from the live URL to verify the health endpoint is up
448. Set up a simple uptime monitor on uptimerobot.com to alert you if the service goes down

🏠 **Home Assignment:** Deploy your Spring Boot API to Render/Railway. Deploy React app to Vercel/Netlify. Both live on the internet!

Day-29 — Capstone Project — Day 1: Design & Backend

📅 02-Apr-26 | 🕐 1:00 PM – 5:00 PM | *Phase 5: Capstone*

Topics Covered

- System design: define entities, REST API contracts, DB schema
- Backend implementation: Spring Boot + Spring Data JPA + MySQL
- Security: JWT auth
- API documentation: Swagger
- Unit tests: JUnit + Mockito
- Git branching strategy for the project
- Code review practices

Lab Manuals

Lab 29.1: System Design & DB Schema

449. Define your capstone app idea (e.g., online store, task manager, library system)
450. Draw an ER diagram showing entities and their relationships
451. Write SQL DDL scripts to create all tables with proper constraints and foreign keys
452. Commit the ERD image and SQL scripts to a new GitHub repo for the capstone project

Lab 29.2: Backend Entities & Repositories

453. Create JPA entity classes mapping to your ER diagram
454. Define Spring Data JPA repositories with required query methods for each entity
455. Write service layer classes with business logic; inject repositories
456. Test service methods with JUnit 5 and Mockito (mock the repository layer)

Lab 29.3: REST Controllers & Security

457. Create REST controllers for all entities with CRUD endpoints
458. Secure all /api/** endpoints with JWT; create /auth/login and /auth/register endpoints
459. Add @Valid validation on all request DTOs
460. Generate Swagger documentation and verify all endpoints are visible in Swagger UI

Lab 29.4: Git Branching & Code Review

461. Create a main branch for production code and a develop branch for integration
462. Create a feature/backend-api branch; push and open a PR to develop
463. Perform a self-code-review: check naming, exception handling, tests, no hardcoded values
464. Merge the PR; tag the commit v1.0.0-backend

🏠 **Home Assignment:** Complete the backend: all entities mapped, REST APIs working, tests passing, pushed to GitHub.

Day-30 — Capstone Project — Day 2: Frontend, Docker & Live Demo

📅 03-Apr-26 | 🕐 1:00 PM – 5:00 PM | *Phase 5: Capstone*

Topics Covered

- React frontend: pages, routing, Redux, API integration
- Styling: Tailwind CSS
- Dockerize both apps + Docker Compose
- Jenkins pipeline: automated build on push
- Deploy to cloud: live URL
- Code walkthrough & demo presentation
- Trainer feedback: code quality, architecture, best practices

Lab Manuals

Lab 30.1: React Frontend Integration

465. Create all required pages in React using React Router v6
466. Integrate RTK Query to call your deployed Spring Boot API
467. Implement login/logout flow with Redux Toolkit auth slice
468. Style all pages with Tailwind CSS; ensure mobile responsiveness

Lab 30.2: Dockerize & Docker Compose

469. Write Dockerfiles for frontend (React + Nginx) and backend (Spring Boot)
470. Create docker-compose.yml combining mysql, backend, and frontend services
471. Test docker compose up --build runs the full stack end-to-end
472. Push Docker configuration files to the GitHub repo

Lab 30.3: CI/CD Pipeline for Capstone

473. Create a Jenkinsfile that: checks out code, builds backend JAR, runs JUnit tests, builds Docker image
474. Trigger the pipeline on every push to the main branch via GitHub webhook
475. Add a stage to push the Docker image to Docker Hub on successful build
476. Add a deploy stage that triggers a re-deploy on your cloud hosting platform

Lab 30.4: Demo Preparation & Presentation

477. Prepare a 10-minute walkthrough: start from the GitHub repo, explain architecture, live demo
478. Demonstrate: registration, login, core CRUD features, logout — all working on the live URL
479. Show the GitHub commit history, PR history, and Swagger docs
480. Present what you would improve next: additional features, performance optimizations, automated E2E tests

🏠 **Home Assignment:** GRADUATION: Live demo of your deployed full-stack app. Share your GitHub repo & live URL.

🎓 **GRADUATION:** Each participant has: a GitHub profile with a deployed full-stack project, Docker skills, CI/CD pipeline.